

Les instructions répétitives

B.U.T. Informatique

Objectifs d'apprentissage

- Savoir ce qu'est une répétitive
- Savoir utiliser une répétitive
- Savoir choisir la bonne répétitive
- Savoir faire une répétitive efficace

Les répétitives

Répétitive

Permet d'exécuter plusieurs fois un bloc d'instructions tant que la réponse à une question est vrai.

- Répétitive, boucle, itérative, . . . que des synonymes
- Permet de répéter plusieurs fois un bloc d'instructions sans avoir à les recopier
- Pas besoin de connaître le nombre d'exécutions de la boucle à l'avance
- Il existe trois types de répétitive

Tant que

Tant que

Tant que une expression booléenne est vrai, je répète un bloc d'instruction. On parle de précondition.

```
while ( x < y ) {  
    z = z * 2;  
    x++;  
}
```

Pour

Pour

Pour une variable allant d'une valeur donnée à une autre, on répète un bloc d'instruction.

```
for ( int i = 0; i < 100; ++i ) {  
    z = z * 2;  
}  
for ( int i = 100; i > 0; —i ) {  
    z = z * 2;  
}
```

Faire... tant que

Faire... tant que

J'exécute un bloc d'instruction et je le répète **tant que** une expression booléenne est vraie. On parle de postcondition.

```
do {  
    z = z * 2;  
    x++;  
} while ( x < y );
```

Du Tant que au Pour

- Le Pour est équivalent au Tant que
- Le Pour contient directement l'initialisation et l'incrément

```
int i = 0;
while ( i < 100 ) {

    z = z * 2;
    i++;

}
```

Est équivalent à

```
for ( int i = 0; i < 100; ++i ) {

    z = z * 2;

}
```

La boucle infinie

- La boucle infinie est une erreur que l'on peut avoir lors de l'exécution
- L'expression booléenne est toujours vrai, et on répète à l'infini le bloc d'instruction
- Lorsque l'on écrit le programme, on doit bien penser à éviter les boucles infinies

Exemple – Erreur courante

```
int i = 0;  
while ( i < 100 ) {  
    z = z * 2;  
}
```

Exemple – Erreur courante 2

```
int i = 0;  
int z = 100;  
while ( i < z ) {  
    z = z * 2;  
    i++;  
}
```

Exemple – Erreur courante 2

```
int i = 0;
int z = 100;
while ( i < z ) {

    z = z * 2;
    i++;

}
```

Quoique, avec le carry (Intro. Archi.), ça devrait s'arrêter

Pour aller plus loin

- En pratique, toutes les boucles ne sont que des boucles `while`
- Le *for each* est un autre type de boucle mais qui n'est pas présent dans tous les langages 🐼
- L'ordinateur n'exécute que les instructions nécessaires pour savoir s'il doit boucler

```
while ( i < z && i != 32) {  
  
    /* Un code */  
  
}
```